

Metamodel Vocabulary Handoff Guide

Use this guide when you need to describe the **shared language** that keeps every Blueprint layer consistent: names, typed identifiers, shared reference meanings, ownership conventions, and other cross-layer vocabulary.

This guide is about the **common language behind all layers**, not about one business slice.

KNOWLEDGE AREA

This guide helps with the **shared language across the blueprint**. Technically, it covers **shared semantic language and reference consistency**.

Guide family map

Where this guide sits in the full Blueprint Handoff Atlas family.

ROOT & CROSS-CUTTING

Handoff Atlas

Blueprint Bundle

Metamodel Vocabulary

Migrations

DESIGN

Architecture

Concepts

Domain

Dynamics

Infrastructure

Interactions

Models

Quality

Rules

Story

GOVERNANCE

Capability

Decisions

Motivation

Organization

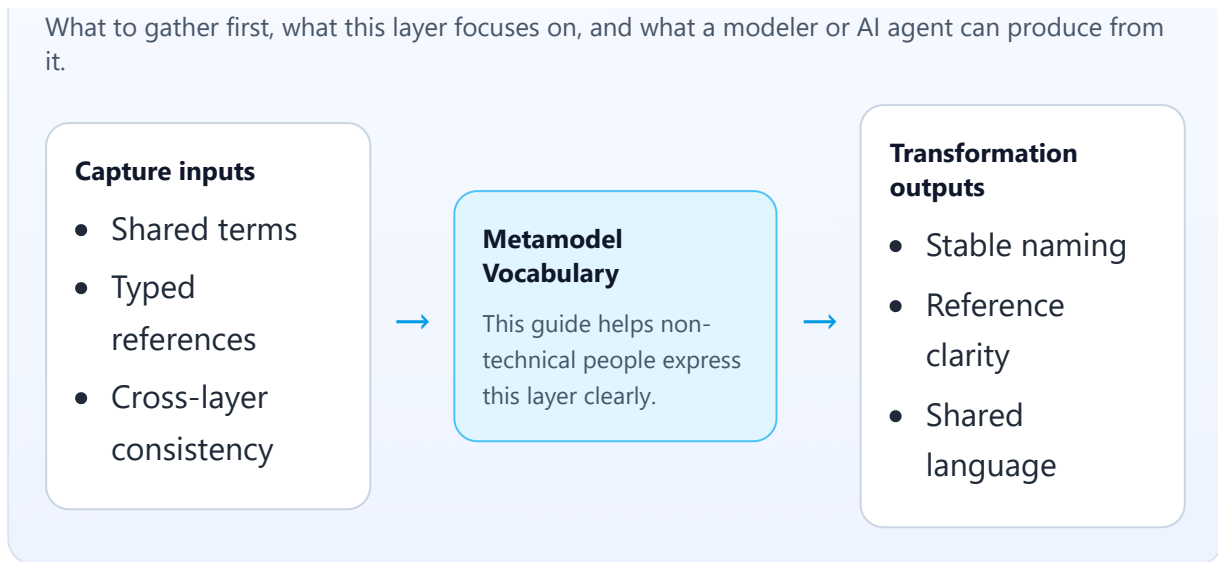
Roadmap

Test Cases

Value Stream

Capture → focus → transformation

What to gather first, what this layer focuses on, and what a modeler or AI agent can produce from it.



What belongs here

- shared vocabulary used across layers
- how important things are named consistently
- how references should stay unambiguous
- what kinds of identifiers or categories matter in business terms
- cross-layer ownership, tags, or version meanings where those shape handoff behavior

What does not belong here

- the full definition of a single business concept or process
- raw schema syntax explanations
- low-level validator mechanics unless they affect author meaning

Core things to capture

- the shared business terms that should be reused everywhere
- the important categories of things the Blueprint distinguishes
- how the same entity should be recognized across multiple guides
- what reference and naming consistency matters to authors
- what cross-layer metadata matters to understanding and reuse

Core relationships to capture

- shared term -> layers that reuse it
- identifier/reference pattern -> entity kind it represents
- ownership/version/tag convention -> content it governs

- vocabulary term -> potentially conflicting term to avoid

Modus operandi

Use this guide when the team needs a shared language:

1. identify terms that appear across many guides
2. align on one meaning for each term
3. distinguish terms that are often confused
4. explain how cross-layer references should stay consistent
5. capture ownership, versioning, or tagging conventions only as author-facing meaning

Prompt set

- what terms appear across multiple guides?
- what does each term mean in one sentence?
- what is often confused with it?
- how do we know two references point to the same thing?
- what naming or ownership conventions must stay stable?

Free-text intake template

- **Shared term or reference kind:**
- **Meaning:**
- **Used in which guides/layers:**
- **What it is often confused with:**
- **Consistency rule for authors:**
- **Ownership/version/tag notes (if relevant):**

Worked example

- **Shared term:** Leave Request
- **Meaning:** an employee's request for time away from work during a defined period
- **Used in:** concepts, story, interactions, rules, tests, roadmap
- **Often confused with:** leave balance, approved leave calendar entry
- **Consistency rule:** use the same name and same primary identity everywhere; do not rename it by screen label or local slang

Layer-specific handoff guidance

When handing off metamodel-vocabulary notes, optimize for **consistency** rather than volume.

A good handoff for this guide should make it easy to see:

- which terms are shared across many layers
- which terms are often confused and must stay distinct
- how the same thing should be named everywhere

If the receiver cannot tell whether two labels refer to the same concept, the handoff is not yet strong enough.

How this becomes YAML

A technical modeler or AI agent can use this guidance to keep IDs, references, names, ownership, and version-related metadata consistent across the full Blueprint model.

Common mistakes

- inventing a different name for the same thing in every guide
- using reference language nobody outside the toolchain understands
- explaining low-level schema mechanics instead of shared author meaning
- treating cross-layer conventions as optional when they are needed for consistency